

Rapport sur un projet C++ Mini Amazon



Module :

Programation Orientée

Objet avec C++

Encadrement :

Pr. Y. Es-Saady

Filière :

Génie Informatique -S4

Membres :

- Oulghazi Fatima Ezzahra
- Nouiri Fatima-Ezzahra
- Elharchaoui Hiba
- Mourtafiaa Soumaya

Les Classes :

- Classe Produit :

```
class Produit {
protected:
    int id;
    string nom;
    double prix;
    string description;
    int stock;
    double note;
    string categorie;
    Statut statut;
    vector<Commentaire> commentaires;
    bool enPromotion;
    double prixOriginal;

public:
    Produit(int id,const string& nom,double prix,
            int stock,const string& categorie,double note=0.0)
        : id(id),nom(nom),prix(prix),stock(stock),
          categorie(categorie),note(note),
          enPromotion(false),prixOriginal(prix),
          statut(stock>0 ? Statut::DISPONIBLE : Statut::RUPTURE_STOCK)
    {
        if(prix<0)          throw invalid_argument("Prix négatif interdit");
        if(stock<0)         throw invalid_argument("Stock négatif interdit");
        if(note<0||note>5)  throw invalid_argument("Note hors [0,5]");
    }
};
```

Produit :

La classe `Produit` représente un article du catalogue de la boutique. Elle regroupe toutes les informations commerciales et logistiques d'un produit : identifiant, nom, description, prix, stock, note, catégorie, statut (`DISPONIBLE` ou `RUPTURE_STOCK`), indicateur de promotion, prix original et liste de commentaires clients.

- Classe Commande :

```
class Commande {
private:
    int        idCommande;
    string      dateCommande, adresseLivraison;
    double      fraisLivraison;
    Paiement*   paiement;
    vector<Produit*> produits;
    vector<int> quantites;

public:
    Commande(int id,const string& date,const string& adresse)
        : idCommande(id),dateCommande(date),
          adresseLivraison(adresse),
          fraisLivraison(25.0),paiement(nullptr){}

    ~Commande(){ delete paiement; }

    void ajouterProduit(Produit* p,int q){ produits.push_back(p); quantites.push_back(q); }

    double calculerTotal() const {
        double t=0;
        for(size_t i=0;i<produits.size();++i)
            t+=produits[i]->getPrix()*quantites[i];
        return t;
    }
    double calculerTotalAvecLivraison() const { return calculerTotal()+fraisLivraison; }
}
```

Commande :

La classe `Commande` représente une commande finalisée passée par un client. Elle conserve l'identifiant de la commande, la date, l'adresse de livraison, les frais de livraison initialisés à 25.0 par défaut, un pointeur vers un objet `Paiement`, ainsi qu'une liste de produits et leurs quantités. Elle contient deux listes parallèles contenant les produits commandés et leurs quantités respectives.

- Classe Panier :

```
class Panier {
protected:
    int        id;           // id dans la table paniers
    int        idClient;
    string      nomClient;
    vector<ArticlePanier> articles;
    string      codePromo;
    double      remisePromo;

public:
    Panier(int id,int idClient,const string& nomClient)
        : id(id),idClient(idClient),nomClient(nomClient),remisePromo(0){}

    virtual ~Panier(){}

    void chargerDepuisDB(Database& db,
                        const vector<Produit*>& tousProduits) {
        // Récupère les infos du panier (code promo, remise)
        auto row = db.fetchOne(
```

La classe `Panier` modélise le panier d'achat d'un client dans un système e-commerce. Elle stocke l'identifiant du panier et du client, le nom du client, une collection d'articles (`vector<ArticlePanier>`), ainsi qu'un code promo et la remise associée. Le constructeur initialise le panier avec les informations du client et met la remise à zéro.

- Classe personne(client) :

```
class Personne {
protected:
    int    identifiant;
    string prenom,nom,email,telephone,motDePasse;
    bool    estConnecte;

public:
    Personne()
        : identifiant(0),prenom(""),nom(""),email(""),
          telephone(""),motDePasse(""),estConnecte(false){}

    Personne(int id,const string& p,const string& n,
              const string& em,const string& tel,const string& pwd)
        : identifiant(id),prenom(p),nom(n),email(em),
          telephone(tel),motDePasse(pwd),estConnecte(false){}

    virtual ~Personne(){
        cout<<"Personne " <<getNomComple()<<" détruite.\n";
    }

    int    getId()          const { return identifiant; }
    string getPrenom()      const { return prenom; }
    string getNom()         const { return nom; }
    string getNomComple()   const { return prenom+" "+nom; }
    string getEmail()       const { return email; }
```

Personne :

La classe Personne est une classe de base représentant tout utilisateur du système. Elle encapsule les informations personnelles telles que l'identifiant, le prénom, le nom, l'email, le téléphone, le mot de passe.

Classe eShop (contient l'ensemble des classes) :

```
1      #include <iostream>
2      #include <string>
3      #include <vector>
4      #include <iomanip>
5      #include <stdexcept>
6      #include <algorithm>
7      using namespace std;
8
9      //. On declare les classes
10     class Produit;
11     class Client;
12     class Paiement;
13     class Commande;
14
```

```

/* =====
CONFIG & STATE
===== */

const API = 'http://127.0.0.1:3000';

let allProducts = [];
let catFilter = '';
let promoFilter = false;
let priceMin = null;
let priceMax = null;
let searchTerm = '';
let currentProductId = null;
let selectedStars = 0;
let pmActive = 'carte';
let regRole = 'client';

/* =====
SESSION
===== */

const s = {
  get() { const u = localStorage.getItem('eshop_u'); return u ? JSON.parse(u) : null; },
  set(u) { localStorage.setItem('eshop_u', JSON.stringify(u)); },
  clear(){ localStorage.removeItem('eshop_u'); },
  ok() { return !!this.get(); }
};

```

Partie JavaScript

```

*,*::before,*::after { box-sizing: border-box; margin: 0; padding: 0; }
html { scroll-behavior: smooth; }
body {
  font-family: 'Jost', sans-serif;
  font-weight: 300;
  background: var(--ivory);
  color: var(--text);
  font-size: 15px;
  line-height: 1.7;
  overflow-x: hidden;
}
h1,h2,h3,h4,h5 { font-family: 'Cormorant Garamond', serif; font-weight: 400; }
a { color: inherit; text-decoration: none; }
img { max-width: 100%; display: block; }
::-webkit-scrollbar { width: 4px; }
::-webkit-scrollbar-track { background: var(--blue-pale); }
::-webkit-scrollbar-thumb { background: var(--blue); }

/* =====
SPLASH SCREEN
===== */

#splash {
  position: fixed; inset: 0; z-index: 9999;
  background: var(--navy);
  display: flex; flex-direction: column;
  align-items: center; justify-content: center;

```

Partie CSS

```

1  <!DOCTYPE html>
2  <html lang="fr">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>eShop</title>
7  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
8  <link href="https://fonts.googleapis.com/css2?family=Cormorant+Garamond:ital,wght@0,300;0,400;0,600;1,
9  <link rel="stylesheet" href="styles.css">
10 </head>
11 <body>
12
13 <!-- =====
14     SPLASH
15     ===== -->
16 <div id="splash">
17   <div class="splash-logo">e<span>Shop</span></div>
18   <div class="splash-tagline">Votre marketplace de confiance</div>
19   <div class="splash_line"></div>

```

Partie HTML

```

1  CREATE DATABASE IF NOT EXISTS eShop_db;
2
3  USE eShop_db;
4
5  CREATE TABLE genres (
6  id      INT          AUTO_INCREMENT PRIMARY KEY,
7  nom     VARCHAR(50)  NOT NULL UNIQUE
8  );
9
10 CREATE TABLE produits (
11 id          INT          AUTO_INCREMENT PRIMARY KEY,
12 nom         VARCHAR(150) NOT NULL,
13 prix        DECIMAL(10,2) NOT NULL CHECK (prix >= 0),
14 prix_original DECIMAL(10,2) NOT NULL,
15 description TEXT,
16 stock       INT          NOT NULL DEFAULT 0 CHECK (stock >= 0),
17 note        DECIMAL(3,2) DEFAULT 0.00 CHECK (note BETWEEN 0 AND 5),
18 categorie   ENUM(
19 'Vêtement','Électronique','Alimentation',
20 'Livre','Sport & Fitness','Maison','Ticket','Ordinateur'
21 ) NOT NULL,
22 statut      ENUM('Disponible','Rupture de stock','Archivé')
23 NOT NULL DEFAULT 'Disponible',
24 en_promotion BOOLEAN      NOT NULL DEFAULT FALSE,
25 image_url   VARCHAR(255)  DEFAULT NULL,
26 created_at  TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
27 updated_at  TIMESTAMP     DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
28 );
29

```

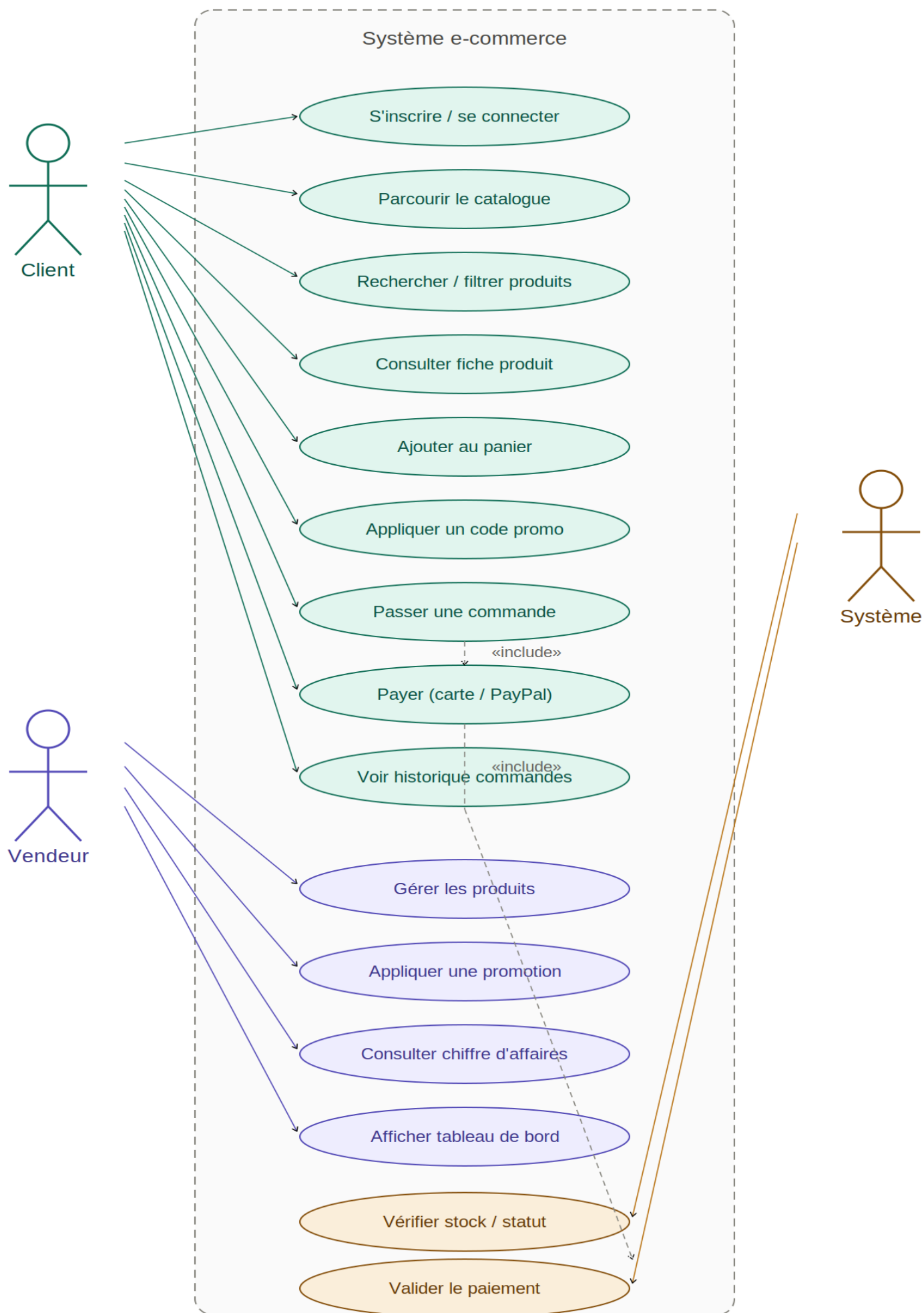
Partie SQL

Languages



● C++ 94.5%
 ● JavaScript 2.6%
 ● CSS 1.6%
 ● HTML 1.3%

Le Diagrammes De Cas D' Utilisation :



LIVRAISON RAPIDE AU MAROC

Découvrez nos meilleures offres

Mode, électronique, sport et bien plus — tout ce dont vous
avez besoin, livré chez vous.

Explorer le catalogue →

Mon Compte



Connexion

Inscription

Email

votre@email.com

Mot de passe

.....

Se connecter

Pas encore inscrit ? [Créer un compte](#)

Connexion

Inscription

Prénom

Ahmed

Nom

BENALI

Email

Ahmed@gmail.com

Téléphone (06/07XXXXXXXX)

0600000000

Mot de passe

.....

Type de compte

Client (Acheteur)



Créer mon compte

Tous les produits

Vêtements

Électronique

Alimentation

Livres

Sport & Fitness

Maison

Tickets

Promotions

PRIX (MAD)

Min



Max



Appliquer

SPORT & FITNESS

Ballon de Football Nike

★★★★★ (0 avis)

199.00 MAD

+ Panier

SPORT & FITNESS

Tapis de Yoga Adidas

★★★★★ (0 avis)

289.00 MAD

+ Panier

MAISON

Canapé Scandinave 3 Places

★★★★★ (0 avis)

3,500.00 MAD

+ Panier

TICKET

Concert Gnawa Festival

★★★★★ (0 avis)

Le Diagramme UML (Diagramme De Classe) :

